Concentric and Archimedes Chess

Version 0
Group 20

GitHub: here

Josef Graf	grafj1@mcmaster.ca	400443948	Project Manager, Developer
Carter Naganobu	naganobc@mcmaster.ca	400471871	Developer
Roland Kim	kimr30@mcmaster.ca	400407536	Developer
Skyler Suk	suks2@mcmaster.ca	400411488	Developer
Harshmeet Singh	singh27@mcmaster.ca	400313448	Developer
Harkirat Singh	singh75@mcmaster.ca	400434064	Developer
Sparsh Poddar	poddas2@mcmaster.ca	400409187	Developer

Complexity

For our project, we developed a client-server model web application which verifies chess rules and manages game state across two custom board variants. Several factors contribute to its high complexity:

1. Variant-specific rules engine: Both Concentric Chess and Archimedes Chess introduce non-standard pieces (prince), twin-cell mechanics, Berolina-style pawn behavior, and variant-specific legal move generation. All of this is encoded as pure deterministic functions that accept the current board state and return either the next state or an illegal-move signal.
2. Real-time multiplayer architecture: The backend uses Cloudflare Workers with Durable Objects to manage persistent WebSocket connections per game session. Server-authoritative validation means every move must round-trip through the server before being applied on the client, requiring careful handling of latency, desync, and race conditions.
3. Board geometry: The circular board required a custom SVG representation with hit-detection logic mapped to irregular tile shapes. Translating ring-and-sector coordinates into valid move sets across both variants demanded significant algorithmic work beyond standard chess logic and resulted in several thousand lines of configuration for the two board formats.
4. State management and replay: Games are archived in a versioned format (variant, initial_setup_id, move_list, timestamps, result, metadata) to allow deterministic replay even if the ruleset evolves, requiring explicit versioning of the rules engine.
5. Full-stack integration: The project spans a Next.js frontend, Cloudflare Workers backend, D1 SQLite database via Drizzle ORM, a shared TypeScript rules library, and real-time WebSocket communication, each layer interacting through well-defined interfaces.

Achievements

Over the course of the project, Group 20 successfully delivered a fully playable, browser-based implementation of two custom chess variants. Key achievements include:

1. Fully functional rules engine for both variants, including all special pieces (prince, crescent, half-moon), twin-cell interactions, and variant-appropriate check/checkmate/stalemate detection.
2. Real-time multiplayer via WebSockets with server-authoritative move validation, desync detection, and automatic client resynchronization.
3. Legal-moves overlay highlighting all valid destinations on piece selection, including variant-specific swing and crescent moves.
4. Complete game history and replay system supporting deterministic reconstruction of any past game from stored move lists.

5. Polished, thematically consistent UI featuring a classic black-and-white chess aesthetic, custom circular SVG board, captured pieces display, and a fully styled game history page.
6. Lobby and invite flow allowing players to share invitation links, confirm readiness, and start games with configurable time controls.
7. In-game rules page providing contextual reference for variant-specific mechanics without interrupting gameplay.

Requirements

P0 Requirements		
Requirement	Description	Completion
Server-authoritative validation	All move validation (including check, checkmate, stalemate, repetition rules, and special piece powers such as king's initial leap, prince movement, crescent/half-moon behavior, twin-cell interactions) is computed server-side; the client is a visualizer only. A move is applied only if the server accepts it.	100%
Deterministic rules engine	The rules for both variants must be encoded as pure, deterministic functions accepting current state → (next state | illegal) to ensure reproducible results and unit testing.	100%
Desync handling	If a client state diverges from the server, the UI must auto-refresh to the last server-committed state and display a non-blocking banner indicating resynchronization	100%
Draw & termination conditions	Server detects 3-fold repetition, 50-move (or variant-appropriate) rule, impossible checkmates, and inactivity timeouts; game is closed with final result stored	100%
Legal-moves overlay	On select, the client must display all legal destinations for the piece (including variant-specific "swing"/twin-cell moves and Berolina-style pawn behavior when on crescents/half-moons).	100%
Single-source clocks (if enabled)	Turn timers are tracked on the server; client clocks are cosmetic mirrors and must tolerate drift correction.	100%

P1 Requirements		
Requirement	Description	Completion

Invite flow w/ lobby	Invitation links open a lobby that shows variant, time controls, and readiness state. The game starts only after both players confirm.	100%
Surrender/Draw workflow	Requests are server-mediated and rate-limited; both are logged with timestamps and result updates.	100%
Game archival format	Store games as {variant, initial_setup_id, move_list, timestamps, result, metadata} such that a replay can be reconstructed deterministically even if the ruleset evolves (versioned rules).	100%
Tutorial & inline hints	Contextual tooltips (e.g., on crescents, twin cells, half-moons) can be toggled; a rules page is available in-game.	50%

Remark: After some consideration, the benefits of the tooltips were not worth the development time implementing, iterating, testing, and maintaining. We decided to defer the information of the tooltips to the rules page which players can easily access during the game.

Highlights & Lowlights

Highlights	Lowlights
• Organizing the visuals for the user interface and related game components was very satisfying when fully completed across multiple iterations, ensuring thematic consistency across the application. • When completed, the game history page provided an excellent and intuitive set of visuals and controls, nearly paralleling well-known chess sites in accessibility and ease of use.	• There were often inconsistencies or edge cases with piece movements and rules (i.e. pawns, prince) that required multiple rounds of testing and debugging to completely resolve. • Creating a SVG file in order to detect tile selection for both boards required a lot of trial and error and sometimes felt misaligned with the visuals of the board up until the final version.

What We Learnt

Building Concentric Chess as a full-stack, real-time multiplayer application surfaced lessons across every layer of the stack:

1. React key stability matters: Using array indices as React keys in dynamic board components caused subtle piece-swap rendering bugs. Composite keys based on position, type, and color are the correct approach.
2. Server-authoritative design pays off: Keeping all game logic on the server made debugging significantly easier there was always one ground truth to reason about, and client desync became a tractable problem.
3. Custom board geometry is genuinely hard: Mapping ring-and-sector coordinates to valid SVG hit regions and then to legal move sets required more geometric reasoning than anticipated. Early abstractions assuming a square grid had to be reworked entirely.
4. Capture detection by count comparison is more robust: Comparing current piece counts against initial counts avoided a class of subtle bugs that arose when diffing board states between moves, especially in multi-capture or promotion scenarios.
5. Cloudflare Workers + Durable Objects have sharp edges: The Durable Objects model is powerful but demands careful attention to serialization, cold-start behavior, and differences between the local Wrangler environment and production.
6. Flat data structures reduce coupling: Returning a flat CapturedPiece[] array from callbacks rather than a nested {white, black} object simplified parent-side filtering and made components easier to compose independently.
7. Test infrastructure deserves early investment: Intermediate test pages (/test-replay, /test-board) were invaluable for isolating functionality from live data. Setting these up earlier would have accelerated debugging throughout the project.